# 校园失物招领系统 - 前端开发指南

## 一、项目概览

### 技术栈

| 技术 | 用途 |
| --- | --- |
| **Vue 3** (Composition API) | 核心框架，使用 `<script setup>` 语法糖 |
| **Vite** | 构建工具和开发服务器 |
| **Vue Router 4** | 前端路由管理 |
| **Pinia** | 状态管理（用户登录态） |
| **Axios** | HTTP 请求客户端 |
| **Element Plus** | UI 组件库（弹窗、选择器、消息提示等） |
| **ECharts 5** | 数据可视化图表 |
| **qrcode** | 二维码生成（分享海报） |
| **html2canvas** | 海报截图保存与分享 |

### 目录结构

```
frontend/
├── index.html              # 入口 HTML
├── vite.config.js          # Vite 配置（含 API 代理）
├── package.json
├── src/
│   ├── main.js             # 应用入口，注册 Vue、Router、Element Plus
│   ├── App.vue             # 根组件（左侧导航栏 + 右侧内容区布局）
│   ├── api/
│   │   └── index.js        # 所有后端 API 封装（Axios 实例 + 拦截器）
│   ├── router/
│   │   └── index.js        # 路由定义 + 导航守卫
│   ├── stores/
│   │   ├── user.js         # 用户状态（登录/退出/用户信息）
│   │   └── theme.js        # 主题切换（亮色/暗黑模式）
│   ├── components/
│   │   ├── NavBar.vue      # 左侧导航栏
│   │   ├── TopBar.vue      # 顶部栏（已弃用）
│   │   └── SharePoster.vue # 分享海报组件
│   └── views/
│       ├── Home.vue         # 首页（物品列表 + 搜索 + 筛选）
│       ├── Login.vue        # 用户登录
│       ├── Register.vue     # 用户注册
│       ├── AdminLogin.vue   # 管理员登录（已重定向到 /login）
│       ├── ItemDetail.vue   # 物品详情页（画廊 + 留言私信）
```

```
│       ├── PublishForm.vue  # 发布/编辑物品表单
│       ├── UserProfile.vue  # 个人中心
│       ├── UserItems.vue    # 我的发布列表
│       ├── UserMessages.vue # 消息中心（聊天室）
│       ├── Dashboard.vue    # 数据看板
│       ├── AdminCertification.vue # 认证审核（管理员）
│       └── AdminUsers.vue   # 用户列表（管理员）
```

---

# 二、核心机制

## 2.1 前后端通信

所有 API 请求通过 `src/api/index.js` 中的 Axios 实例统一管理：

```
const request = axios.create({
    baseURL: '/api',       // 所有请求以 /api 开头
    timeout: 15000
})
```

**代理机制**：Vite 开发服务器将 `/api` 和 `/uploads` 请求转发到后端 Tomcat（默认 `http://localhost:8080`），`vite.config.js` 配置：

```
proxy: {
    '/api':     { target: 'http://localhost:8080', changeOrigin: true },
    '/uploads': { target: 'http://localhost:8080', changeOrigin: true }
}
```

**认证方式**：JWT Token 存储在 `localStorage`，请求拦截器自动在 `Authorization` 头中添加 `Bearer <token>`。

**响应处理**：

- 拦截器自动提取 `res.data`（即 `{ code, message, data }` 中的 data）
- `code !== 200` 时自动弹错误提示
- `401` 时自动清除 token 并跳转登录页

## 2.2 用户认证流程

```
登录页 → POST /api/auth/login { username, password }
       ← { code:200, data: { token: "xxx", user: {...} } }
       → localStorage.setItem('token', token)
       → userStore.setUserInfo(user)
       → router.push('/')
```

后续所有需认证的请求自动带 Token，路由守卫 `router.beforeEach` 拦截未登录者。

### 2.3 消息/聊天功能

**发送留言**（从物品详情页）：

```
POST /api/messages { receiverId, itemId, content }
```

后端兼容驼峰和下划线命名（`receiverId` / `receiver_id`），插入后回查完整消息对象（含发送者昵称、物品标题）返回前端。

**消息中心**（`UserMessages.vue`）：

1. `onMounted` 同时请求 `/api/messages/user/received` 和 `/api/messages/user/sent`
2. 合并去重后按 `otherId`（对方用户ID）分组为**会话列表**
3. 点击会话 → 筛选该对方的全部消息按时间排序显示
4. 自动标记未读消息为已读（`PUT /api/messages/{id}/read`）

**关键修复历史**：

- 后端字段名从下划线改为驼峰输出（`buildMessageMap` 方法）
- 消息发送参数兼容驼峰/下划线（`receiverId` vs `receiver_id`）
- getMyMessages/getSentMessages 的 SQL 全部 JOIN 两张用户表 + LEFT JOIN items 获取物品标题

### 2.4 物品发布流程

```
PublishForm → 上传图片: POST /api/upload (multipart/form-data)
            ← { url: "/uploads/uuid.png" }
            → 图片预览区即时显示

            → 发布: POST /api/items { type, categoryId, title, description,
location, contact, images }
              (images 为 JSON.stringify 后的字符串数组)
            ← { id: 新物品ID }
            → 自动触发智能匹配: GET /api/items/match/{id}
            → 如有匹配结果弹出弹窗
```

### 2.5 图片上传与显示链路

```
前端选文件 → FormData → POST /api/upload
            → UploadServlet 保存到 web/uploads/
            ← 返回 { url: "/uploads/uuid.png", fileName: "uuid.png" }

前端存储: imageList.value.push(url)
提交时: images: JSON.stringify(imageList)

后端读取: images = (String) body.get("images") → 直接存 DB
```

```
页面展示: JSON.parse(item.images) → 数组 → arr[0] → <img :src="url">
          Vite 代理: /uploads → http://localhost:8080 → Tomcat 提供静态文件
```

### 2.6 数据看板

`Dashboard.vue` 同时请求两个公开接口：

- `GET /api/items/dashboard/stats`：返回 totalItems、todayNew、resolvedCount、resolveRate、dailyTrend（近7天数据）、categoryDistribution（含分类名称）
- `GET /api/auth/reputation/rank?top=10`：返回信誉积分 Top 10 用户

后端 SQL 聚合计算所有统计数据，直接返回前端渲染 ECharts 图表。

### 2.7 暗黑模式

`stores/theme.js` 管理暗黑模式状态，通过给 `<html>` 添加/移除 `dark` class 实现。所有页面通过 `html.dark .xxx` 选择器覆盖暗色样式。

### 2.8 二维码分享海报（创新点✨）

分享海报功能通过 `SharePoster.vue` 组件实现，支持一键生成带二维码的海报并保存/分享，是系统的创新特色功能。

**触发入口**：物品详情页（`ItemDetail.vue`）中的「分享海报」按钮。

**数据流**：

```
ItemDetail.vue 点击「分享海报」
  → SharePoster.vue 接收 item 数据，visible = true
  → watch visible 触发 generateQRCode()
  → 动态 import('qrcode') 生成二维码 DataURL
  → 二维码渲染到海报卡片中
  → 用户可「保存图片」或「分享」
```

**二维码生成**（`generateQRCode`）：

```
async function generateQRCode() {
    const QRCode = (await import('qrcode')).default
    const url = `${window.location.origin}/#/item/${props.item.id}`
    qrCodeUrl.value = await QRCode.toDataURL(url, {
        width: 120, height: 120, margin: 1,
        color: { dark: '#4a4a6a', light: '#ffffff' }
    })
}
```

- 动态导入 `qrcode` 库，减少首屏加载体积
- 生成指向物品详情页的链接二维码：`/item/{id}`

- 自定义颜色与系统主题色（`#4a4a6a`）一致

**海报保存**（`handleSave`）：使用 `html2canvas` 将海报 DOM 截图 → 生成 PNG 下载

**海报分享**（`handleSharePoster`）：

- 优先使用 Web Share API (`navigator.share`) 原生分享（含图片文件）
- 降级方案：自动保存到本地

**依赖库**：

- `qrcode`：二维码生成
- `html2canvas`：DOM 截图转图片

---

# 三、开发与调试

## 启动前端

```
cd frontend
npm install
npm run dev
# 默认 http://localhost:5173
```

## 常见问题排查

1. **API 请求 404**：确认后端在 `8080` 端口运行
2. **图片不显示**：检查 Vite 是否重启（代理配置需重启生效）、`web/uploads/` 目录是否有文件
3. **登录失败**：检查 MySQL 服务是否启动，数据库连接配置是否正确
4. **图表不渲染**：确保 `echarts` 库已安装，ref 绑定正确